Versi 2.1

Agustus 2026

PEMROGRAMAN MOBILE

MODUL 3 - PEMROGRAMAN ASYNCHRONOUS & API CALL

Penulis:

Eko Felix Nur Noftianto, M.Kom.

Wira Yudha Aji Pratama

Fatahillah Al-Fatih

LABORATORIUM INFORMATIKA

Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI **2**
PENDAHULUAN **3**
1. TUJUAN 3
2. TARGET MODUL 3
3. PERSIAPAN 3
4. KATA KUNCI 3
MATERI **4**
1. SYNCHRONOUS VS ASYNCHRONOUS 4
1.1. DEFINISI UTAMA 4
1.2. ASYNC FUNCTION 6
1.3. PROMISE & AWAIT 7
1.4 .THEN() 13
2. API CALL 14
3. PRAKTIK 18
4. CODELAB-1 (10-15min, kerjakan di lab) 22
TUGAS PEKAN DEMO **23**
1. DESKRIPSI 23
2. KRITERIA PENILAIAN 24
3. SKALA PENILAIAN 24
KESIMPULAN AKHIR MODUL **25**

PENDAHULUAN

1. TUJUAN

1. Memungkinkan mahasiswa memahami konsep dasar pemrograman synchronous dan asynchronous.
2. Memberikan kepada mahasiswa pemahaman yang jelas mengenai fungsi asynchronous, Promise, async/await, dan .then().
3. Membekali mahasiswa dengan pengetahuan yang diperlukan untuk melakukan API call dan menangani data asynchronous dalam aplikasi mobile.

2. TARGET MODUL

1. Mahasiswa mampu membedakan pemrograman synchronous dan asynchronous.
2. Mahasiswa mampu mengimplementasikan fungsi asynchronous menggunakan async/await dan .then().
3. Mahasiswa mampu memahami dan menangani Promise dalam JavaScript/TypeScript.
4. Mahasiswa mampu melakukan API call dan memproses data yang dikembalikan oleh API.

3. PERSIAPAN

1. Mahasiswa Muslim dapat membaca doa berikut sebelum memulai pembelajaran:

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop atau komputer pribadi (Windows/Mac).
3. Visual Studio Code.
4. Koneksi internet.
5. Telepon seluler (Android / iOS).
6. Motivasi dan komitmen yang kuat untuk belajar.

4. KATA KUNCI

Synchronous Programming, Asynchronous Programming, Async Function, Promise, async/await, .then(), API Call

MATERI

Pada Modul 3, kita akan beralih ke topik yang lebih lanjut dan juga sangat penting. Kita akan mempelajari bagaimana aplikasi yang kita gunakan sehari-hari mengambil data dari server. Kamu mungkin juga akan mempelajari konsep ini dalam mata kuliah **Web Programming**, jadi dalam modul ini kita akan membahas dasar-dasarnya terlebih dahulu. Jika kamu tertarik, silakan pelajari topik ini lebih lanjut di luar modul.

1. SYNCHRONOUS VS ASYNCHRONOUS

1.1. DEFINISI UTAMA

Synchronous programming adalah pendekatan pemrograman di mana tugas dijalankan **satu per satu** dan **secara berurutan**, yang berarti program harus menyelesaikan tugas saat ini terlebih dahulu sebelum melanjutkan ke tugas berikutnya.

Contoh:

example.tsx

```
console.log("First");
console.log("Second");
console.log("Third");
```

Output:

```
first
second
third
```

Sementara itu, **asynchronous programming** memungkinkan program untuk memulai sebuah tugas yang **mungkin membutuhkan waktu untuk selesai** dan **tetap menjalankan kode lainnya** tanpa harus menunggu tugas tersebut selesai. Setelah tugas asynchronous selesai, program dapat menangani hasilnya nanti.

Contoh:

example.tsx

```tsx
console.log("First");
setTimeout(
  () => { console.log("Second"); },
  1000
); // This means to run the command but wait for 1000ms (1 second) first.
console.log("Third");
```

Output:

```
first
third
```

Lalu setelah 1 detik:

```
first
third
second
```

Di sini, setTimeout() merupakan contoh **operasi asynchronous**. Perilakunya bersifat *asynchronous* karena *callback*-nya dijalankan kemudian. **Namun**, setTimeout() sendiri **bukan merupakan async function**. Sebuah fungsi disebut *async function* jika dideklarasikan menggunakan kata kunci ***async***.

1.2. ASYNC FUNCTION

*Sekarang mari kita lihat apa perbedaan antara sebuah **async function** dan **normal function**.*

Normal Function:

example.tsx

```tsx
const example = (delay: number) => {
    let x;
    setTimeout(
        () => { x = "done"; }, delay
    );
    return x;
};

console.log(example(1000)); // guess the output
console.log(typeof example(1000)); // guess the output
```

Di sini, ***example* bukan merupakan async function**, meskipun di dalamnya terdapat operasi asynchronous. Menurutmu, apa output dari kode di atas? **"Done"** dan **String??** Salah.

Output yang benar adalah **undefined** dan **undefined**:

```
undefined
undefined
```

Karena ketika program dijalankan, perintah **x = "Done"** belum dijalankan (ditunda selama 1 detik), tetapi program sudah langsung melanjutkan ke kode berikutnya dan menjalankan **return x**, yang pada saat itu masih bernilai **undefined**. Kemudian, fungsi *console.log* dijalankan. Nilai **x** memang baru akan diubah menjadi **"done"** setelah 1 detik di *background*.

Sekarang mari kita lihat apa yang terjadi jika kita menambahkan kata kunci ***async*** pada deklarasi fungsi:

example.tsx

```tsx
const example = async (delay: number) => {
    let x;
    setTimeout(
        () => { x = "done"; }, delay
    );
    return x;
};

console.log(example(1000));
console.log(typeof example(1000));
```

Output:

```
Promise { undefined }
object
```

Type tersebut berubah dari **undefined** menjadi sebuah **object**. Mengapa? Karena ketika kita menambahkan kata kunci **async**, TypeScript memperlakukan nilai yang dikembalikan oleh fungsi tersebut sebagai sebuah **Promise object**.

1.3. PROMISE & AWAIT

Apa itu object *Promise*? Object *Promise* adalah sebuah *object* yang dapat kita gunakan untuk **merepresentasikan sesuatu yang hasilnya masih dalam proses**.



Image 1.3.1

Daripada program hanya mengatakan, ***"Tidak ada apa-apa!"***, program dapat dengan sopan mengatakan, ***"Tunggu sebentar ya. Apa yang kamu minta membutuhkan waktu untuk diproses. Tapi saya berjanji akan memberikan sesuatu nanti."***

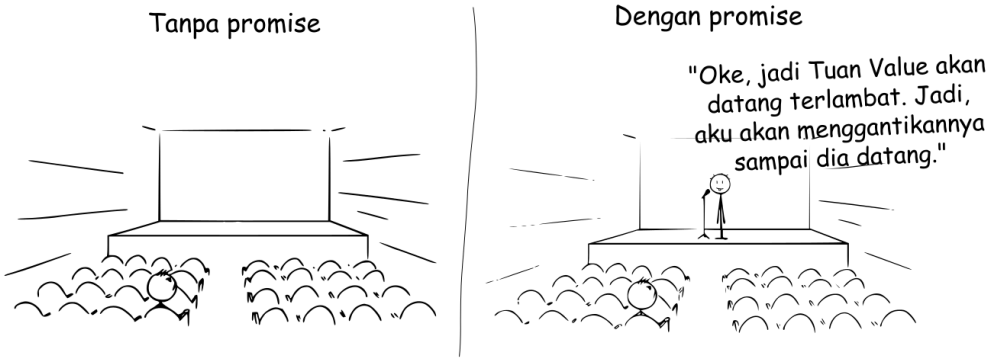


Image 1.3.2

Lalu, mengapa isi *Promise object* masih berupa ***undefined***? Karena kita harus menunggu 1 detik sebelum datanya siap. Namun, program tersebut tidak melakukannya. Program justru langsung menampilkan isi *Promise* yang masih berupa *undefined*, kemudian memberikan nilai **x** menjadi **"done"** setelahnya.

Jadi, pertanyaannya adalah: ***"Bagaimana kita memberi tahu sebuah fungsi untuk menunggu operasi asynchronous selesai sebelum melanjutkan?"***

Kita akan menggunakan kata kunci ***await***.

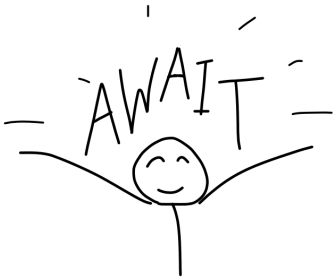


Image 1.3.3

Kata kunci *await* adalah kata kunci yang **digunakan untuk menunggu hingga isi sebuah Promise selesai terlebih dahulu sebelum melanjutkan ke baris kode berikutnya.**



Image 1.3.4

Namun, sebelum itu, kita perlu melakukan sedikit modifikasi pada ***example()*** terlebih dahulu. Kita perlu menggunakan **resolve()** daripada hanya menggunakan **return** seperti pada contoh sebelumnya.

Untuk melakukannya, kita akan membuat ***Promise object*** itu sendiri menggunakan ***Promise constructor***, daripada mendapatkan *Promise object* dari sebuah ***async function.***

Promise constructor adalah *constructor* yang memungkinkan kita **membuat sebuah *Promise object*** menggunakan pola: `new Promise(executor)`.

Kamu juga dapat memberikan ***generic*** type pada sebuah Promise seperti: `Promise<string>(executor)`. Artinya, ***Promise*** tersebut nantinya akan menghasilkan tipe ***string***. Namun, kamu hanya dapat melihat informasi ini di *code editor*. Pada output, kamu tetap harus menunggu fungsi selesai terlebih dahulu sebelum melihat hasilnya. Hal ini *opsional*, tetapi akan sangat berguna ketika kamu sudah lebih sering menggunakannya.



Image 1.3.5

Oke, *Promise* di sini memiliki 2 fungsi yang dapat kita gunakan dan diberikan melalui argumen ***executor***, yaitu: ***resolve()*** dan ***reject()***. Jadi, secara sederhana, bentuknya dapat terlihat seperti ini: `new Promise( (resolve, reject) => {someAsyncOperation} )`.

Kita dapat menggunakan **resolve()** atau **reject()**. Kita menggunakan ***resolve()*** ketika operasi **berhasil** dan ***reject()*** ketika operasi **gagal**.

Mari kita lihat contohnya:

example.tsx

```tsx
const example = new Promise(
    (resolve, reject) => {
        setTimeout(
            () => {isSuccess ? resolve("Succeed") : reject("Failed");},
            5000
        );
    }
);
```

Anggap saja dalam contoh di atas, ***setTimeout*** adalah fungsi yang bertanggung jawab untuk mengambil data dari server (anggap saja seperti itu). Server tersebut lambat, sehingga proses pengambilan data membutuhkan waktu sekitar 5 detik.

Jika data berhasil diambil dari server dalam waktu 5 detik tersebut, ***isSuccess*** akan bernilai **true** dan resolve akan dipanggil dengan pesan "**Succeed**".

Namun, jika 5 detik telah berlalu dan data **masih tidak dapat** diambil dari server (misalnya karena server sedang *down*), ***isSuccess*** akan bernilai **false** dan reject akan dipanggil dengan pesan "**Failed**".

Hal ini berbeda dengan **return** sebelumnya.

Contoh 1 (tanpa await):

example.tsx

```
const example = new Promise(
    (resolve, reject) => {
        setTimeout(
            () => {isSuccess ? resolve("Succeed") : reject("Failed");},
            5000
        );
    }
);

console.log(example); // output: Promise { <pending> }
console.log(typeof example); // output: object
```

Seperti yang dapat kamu lihat, sekarang bukan lagi *Promise { undefined }* karena kita sudah menggunakan **resolve()**.

Contoh 2 (dengan await):

example.tsx

```
const example = new Promise(
    (resolve, reject) => {
        setTimeout(
            () => {isSuccess ? resolve("Succeed") : reject("Failed");},
            5000
        );
    }
);

await example;
console.log(example); // output: Promise { 'Succeed' }
console.log(typeof example); // output: object
```

Ketika kode dijalankan, output akan muncul setelah kamu menunggu selama **5000 ms (5 detik)**.

Contoh 3 (dengan await & assignment):

example.tsx

```tsx
const example = new Promise(
    (resolve, reject) => {
        setTimeout(
            () => {isSuccess ? resolve("Succeed") : reject("Failed");},
            5000
        );
    }
);

const result = await example;
console.log(result); // output: Succeed
console.log(typeof result); // output: string
```

Contoh 4 (kamu tidak dapat melakukannya seperti ini, harus menggunakan resolve()):

example.tsx

```tsx
const example = async () => {
    let x;
    setTimeout( () => { x = "Succeed" }, 5000 );
    return x;
}

const result = await example;
console.log(result); // output: [AsyncFunction: example]
console.log(typeof result); // output: function
```

Namun, ya, itu hanyalah sebuah **analogi**. Penggunaan dalam dunia nyata biasanya akan terlihat seperti ini:

example.tsx

```tsx
function getUser() {
    return new Promise((resolve, reject) => {
        fetch("https://example.com/api/user")
        .then(response => {
          if (response.ok) { resolve(response.json()); }
          else { reject("Failed to get user"); }
        });
    });
}
```

Nah, inilah yang akan kita pelajari pada **Modul 3**, yaitu **API Calls**.

Jadi, ketika **getUser()** dipanggil, fungsi tersebut langsung mengembalikan *Promise*. Kemudian, ***fetch()*** meminta data dari API dengan melakukan **HTTP request**. Jadi, dalam kode ini, kita memiliki sebuah *Promise* di dalam *Promise* lainnya.

Sekarang, mari kita lanjutkan ke konsep **.then()**. Apa itu **.then()**?

1.4 .THEN()

.then() adalah *method* dari *Promise* yang digunakan untuk mengatakan: *"Jika Promise ini selesai dengan sukses, jalankan kode ini."*

Promise sebelumnya menghasilkan sebuah nilai, dan nilai tersebut kemudian diteruskan ke **callback .then()** melalui *parameter* yang secara default disebut **result**.

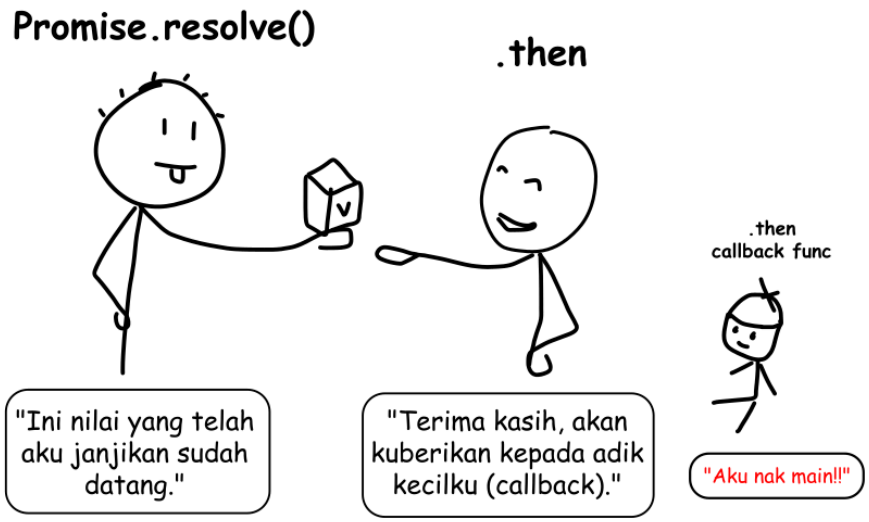


Image 1.4.1

.then() sendiri juga menghasilkan sebuah *Promise* baru, dan nilai dari *Promise* tersebut berasal dari apa pun yang dikembalikan (*returned*) oleh *callback*.



Image 1.4.2

Contoh:

example.tsx

```tsx
const example = Promise.resolve(10);

example
.then(result => { return result * 2; })
.then(result => { console.log(result); }); // output: 20
```

Baiklah, mari kita bahas hal ini pada Bab 2 di bawah karena ini merupakan topik yang berbeda. WKwk.

Jadi, kesimpulannya, dalam sebagian besar aplikasi, ketika server sedang lambat, kita masih dapat melakukan hal-hal lain selain hanya menunggu data yang kita minta selesai **diambil (*fetching*)**.

Hal ini karena kita menggunakan **pemrograman asynchronous**, yang memungkinkan JavaScript tetap menjalankan kode lainnya tanpa menghentikan seluruh proses hanya karena sedang menunggu data dari server.

2. API CALL

Baiklah, mari kita mulai dengan definisi **API**. Jadi, apa itu **API**?

API (*Application Programming Interface*) adalah jembatan yang memungkinkan berbagai aplikasi atau sistem untuk saling berkomunikasi dan bertukar data.

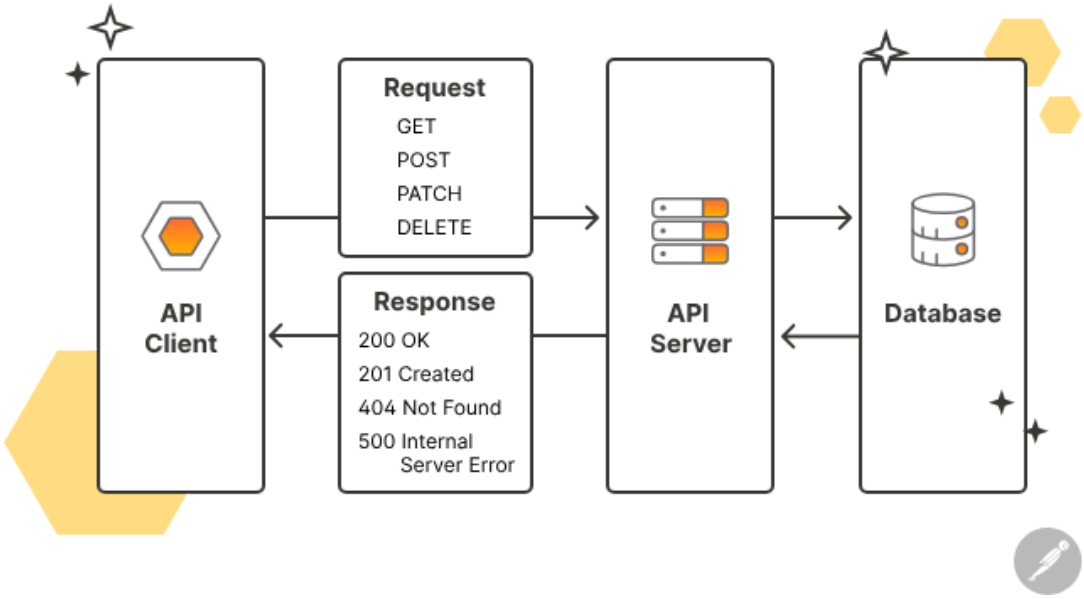


Image 2.1

Daripada menggunakan data statis atau *hardcoded* di dalam aplikasi, kita dapat mengambil data dari sumber eksternal melalui API. Sebagai contoh, kita dapat mengambil data seperti informasi cuaca, informasi film, atau informasi negara dari API publik gratis yang tersedia di internet untuk keperluan pembelajaran dan latihan. **Contoh penggunaan pada kode:**

apicall.tsx

```tsx
import { useEffect, useState } from "react";
import { View, Text } from "react-native";

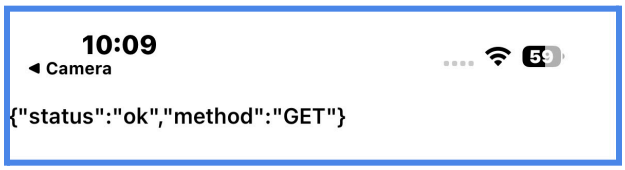
export default function App() {
    const [data, setData] = useState<any>(null);

    useEffect(() => {
        const fetchData = async () => {
            const response = await fetch( "https://dummyjson.com/test" );
            const result = await response.json();
            setData(result);
        };
        fetchData();
    }, []);

    return (
        <View>
            <Text>{JSON.stringify(data)}</Text>

        </View>
    );
}
```

Output:



Namun, sebelum kita mulai mengimplementasikan kode tersebut, ada beberapa konsep dasar yang perlu kamu pahami terlebih dahulu.

Bagi kalian yang baru pertama kali mempelajari konsep ini, mungkin kalian bertanya-tanya: *Untuk apa ini digunakan? Mengapa kita membutuhkannya? Mengapa ada begitu banyak hal yang harus dipelajari sebelum kita benar-benar bisa melakukan API call?*

Jangan khawatir, kita akan mulai dari dasar. Namun, jika kalian sudah pernah mempelajari konsep-konsep ini sebelumnya, kalian cukup mengikuti pembahasannya karena sudah familiar dengan konsep-konsep tersebut.

Dasar dari API call (setidaknya dengan cara yang akan kita gunakan dalam modul ini) adalah konsep **HTTP Request**.

Ingat ketika kalian mempelajarinya pada praktikum **Komunikasi Data** di semester 3? Atau sudah lupa? 😁

Jadi, apa itu HTTP Request?

HTTP Request adalah **pesan yang dikirim oleh client** (seperti aplikasi mobile, web browser, atau website) kepada server untuk meminta data atau melakukan tindakan tertentu.

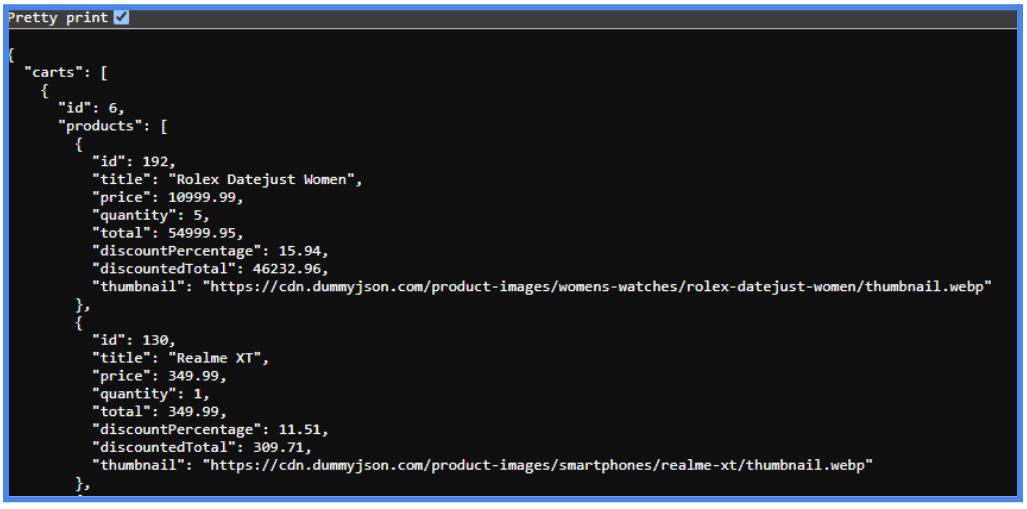
Sebagai contoh, ketika kamu membuka *Instagram*, *YouTube*, atau *Tokopedia*, aplikasi tersebut mengirimkan HTTP Request kepada server untuk meminta informasi seperti *user profile*, *video*, *produk*, *komentar*, dan berbagai jenis data lainnya.

Server kemudian memproses request tersebut dan mengirimkan kembali **response** yang berisi informasi yang diminta.

Baiklah, sekarang saya ingin kamu membuka tautan berikut di web browser:

https://dummyjson.com/users/6/carts

(Semoga tautannya masih berfungsi dan belum di-deprecated oleh server 😁). Kamu mungkin akan diarahkan ke halaman yang terlihat kurang lebih seperti ini:



Wow, jangan panik dulu. Apa yang kamu lihat adalah sebuah ***JSON response***.

Data dari tautan tersebut adalah data yang akan kita ambil dan tampilkan dalam aplikasi.

Mengapa kita melakukan hal tersebut?

Agar data kita tidak lagi disimpan **dalam kode**. Sebaliknya, data tersebut berada di luar aplikasi, yaitu dalam sebuah database di server.

Artinya, ketika data berubah atau ketika kita ingin melakukan tindakan tertentu, kita tidak selalu perlu mengubah kode aplikasi. Kita cukup berkomunikasi dengan server melalui HTTP Request.

Itulah penjelasan sederhananya.

HTTP Request tidak hanya digunakan untuk mengambil data. Terdapat berbagai HTTP method yang tersedia, seperti:

- **GET** → mengambil data
- **POST** → membuat/menambahkan data baru
- **DELETE** → menghapus data
- **PUT** → mengganti/memperbarui seluruh resource yang sudah ada
- **PATCH** → memperbarui sebagian field tertentu dari sebuah resource

Itulah beberapa method yang paling umum digunakan.

Namun, dalam modul ini, kita hanya akan berfokus pada **GET request** (*mengambil data*). Method lainnya akan dibahas pada praktikum lain karena berkaitan erat dengan database dan pengembangan backend.

Sekarang, URL yang kamu gunakan sebelumnya untuk mengambil data biasanya disebut sebagai ***endpoint***.

Response yang kamu terima dapat berbeda-beda tergantung pada endpoint dan parameter yang kamu kirimkan.

Sebagai contoh: https://dummyjson.com/users/6/carts mengembalikan informasi tentang keranjang belanja (*shopping carts*) milik user nomor 6.

Sedangkan https://dummyjson.com/users/10/carts mengembalikan informasi tentang keranjang belanja milik user nomor 10.

Proses mengambil data dari API biasanya disebut sebagai ***fetching***.

Data JSON yang dikembalikan oleh server disebut **respons**.

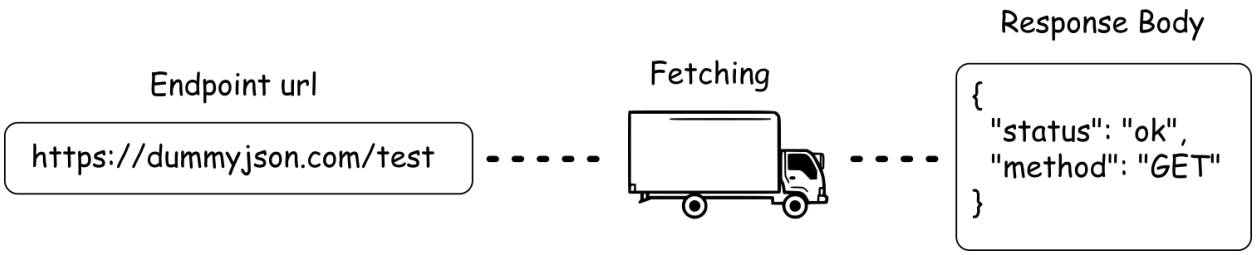


Image 2.2

Nantinya, response ini akan disimpan ke dalam struktur data yang telah kita siapkan, biasanya berupa *Array of Objects*, yang telah kita pelajari pada Modul 1.

Sebenarnya, ketika mengirim HTTP Request, kita juga dapat menyertakan informasi tambahan seperti:

- **Body** → digunakan untuk mengirim data ke server (umumnya digunakan pada request *POST*, *PUT*, dan *PATCH*)
- **Token** → digunakan untuk autentikasi dan otorisasi
- **Headers** → digunakan untuk mengirim metadata tambahan seperti *content type*, preferensi bahasa, atau informasi autentikasi

Namun, jangan khawatir tentang hal tersebut untuk saat ini 😁. Kalian mungkin akan mempelajarinya secara lebih mendalam dalam mata kuliah **Web Programming** atau **Backend Development**, bukan dalam modul ini.

3. PRAKTIK

Cukup dengan teori. Sekarang mari kita terapkan semua yang telah kita pelajari ke dalam praktik dengan membuat project baru dan menerapkan konsep-konsep tersebut secara langsung.

Di **root folder** project-mu, buat file dan folder berikut:

example.txt

```
anime-hunter/
├── app/
│   ├── _layout.tsx
│   ├── index.tsx
│   ├── search.tsx
│   ├── favorites.tsx
│   └── character/
│       └── [id].tsx
│
├── components/
│   ├── CharacterCard.tsx
│   ├── Loading.tsx
│   ├── ErrorMessage.tsx
│   └── StatCard.tsx
│
├── services/
│   ├── api.ts
│   ├── characterService.ts
│   └── favoriteService.ts
│
├── types/
│   ├── character.ts
│   └── api.ts
│
└── utils/
    └── delay.ts
```

Laboratorium INFORMATIKA Universitas Muhammadiyah Malang

1. **types/character.ts**
    a. Buka file types/character.ts dan isi file tersebut dengan kode dari tautan berikut: Tautan. Untuk menjaga agar *type* tetap terorganisir dan dapat digunakan kembali, project berukuran besar biasanya memiliki folder khusus untuk menyimpan *type* yang digunakan. Hal ini membuat kode lebih rapi dan memungkinkan *type* yang sama digunakan kembali di berbagai file.

2. **types/api.ts:**
    a. Isi file ini dengan kode sederhana berikut:

_layout.tsx

```
export interface ApiResponse<T> {
  success: boolean;
  data: T;
  message: string;
}
```

3. **utils/delay.ts**
    a. Isi file ini dengan kode sederhana berikut:

_layout.tsx

```
export function delay(ms: number): Promise<void> {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}
```

Project berukuran besar juga biasanya memiliki folder utils yang berisi fungsi-fungsi *utility* yang dapat digunakan kembali. Fungsi-fungsi ini biasanya dirancang untuk menerima berbagai *argument* sehingga dapat digunakan kembali dalam konteks dan situasi yang berbeda, daripada mendeklarasikan fungsi yang sama berulang kali di file yang berbeda. Namun, dalam project ini kita hanya memiliki satu fungsi *utility*. 😄

4. **services/api.ts:**
    a. Kita akan membuat fungsi-fungsi untuk mengambil data (*fetching data*). Namun, daripada menggunakan API eksternal, kita akan membuat data kita sendiri (ini mensimulasikan fetch data external). Jika kamu melihat kodenya, kamu akan menemukan data anime, rating anime, dan data karakter anime. Salin kode dari tautan ini.

5. **services/characterService.ts:**
    a. Di file ini, kita sengaja menerapkan beberapa konsep yang telah kita pelajari sejauh ini. Salin kode dari tautan berikut: Tautan:

6. **service/favoriteService.ts:**
    a. Pertama, instal package yang diperlukan menggunakan terminal: `npm expo install @react-native-async-storage/async-storage`

b. Kemudian, salin kode dari tautan ini. Kita akan mempelajari lebih lanjut tentang hal ini pada modul berikutnya, yaitu Modul 4, saat kita membahas *local storage*.

7. **components/Loading.tsx:**
   a. Sekarang, mari kita masuk ke folder /components. Folder components biasanya berisi komponen yang dapat digunakan kembali di berbagai file. Sebagai contoh, bayangkan kamu ingin membuat fitur pencarian dan lima halaman berbeda yang perlu menggunakan komponen pencarian yang sama. Daripada membuat komponen yang sama sebanyak lima kali, kamu dapat membuatnya sekali di dalam folder components dan menggunakannya kembali di mana pun diperlukan. Hal ini juga membantu menjaga file utama, seperti index.tsx, tetap rapi dan lebih mudah dibaca.
   b. File ini akan berisi komponen yang menampilkan indikator *loading*. Isi file tersebut dengan kode dari tautan ini.

8. **components/ErrorMessage.tsx:** Link.

9. **components/CharacterCard.tsx:** Link.

10. **components/StatCard.tsx:** Link.

11. **app/_layout.tsx:**
    a. Kita sudah mempelajarinya di Modul 2, bukan? File ini memungkinkan kita membuat komponen atau *layout* yang memengaruhi semua halaman dalam struktur aplikasi yang sama. File ini sangat berguna untuk hal-hal seperti *header*, *footer*, bilah navigasi, dan *layout* bersama lainnya. Ikuti kode yang disediakan di bawah ini:

_layout.tsx

```tsx
import { Stack } from "expo-router";

export default function RootLayout() {
  return (
    <Stack>
      <Stack.Screen name="index" options={{ title: "Anime Hunter"}}/>
      <Stack.Screen name="search" options={{title: "Search Character"}}/>
      <Stack.Screen name="favorites" options={{title: "Favorites"}}/>
      <Stack.Screen name="character/[id]" options={{title: "Character Detail"}}/>
    </Stack>
  );
}
```

12. **app/index.tsx:**
    a. Ini akan menjadi *landing page* kita. Karena file ini berada di dalam folder /app dan bernama index.tsx, Expo Router akan memperlakukannya sebagai halaman default pada aplikasi kita. Salin kode dari tautan ini.

13. **app/search.tsx:**
    a. Halaman ini memungkinkan kita mencari karakter berdasarkan nama. Setelah itu, kita dapat melihat detail karakter tersebut dan menekan ikon 🖤 untuk menambahkan karakter ke daftar favorit kita. Salin kode dari tautan ini.

14. **app/character/[id].tsx:**
    a. Halaman ini berisi informasi terperinci tentang karakter yang kita pilih. Halaman ini juga menerapkan *dynamic routing*, seperti yang telah kita pelajari pada modul sebelumnya. Salin kode dari tautan ini.

15. **app/favorite.tsx:** Link.

4. CODELAB-1 (10-15min, kerjakan di lab)

Codelab ini **sederhana**. Cukup ikuti langkah-langkah pada ***Bab 3*** di atas.

TUGAS PEKAN DEMO

1. DESKRIPSI

1. **Buat kelompok & pilih tema:**
    a. Buat kelompok yang terdiri dari 3 anggota dan pilih tema project kalian sendiri melalui tautan *di sini*. *(Tema project harus unik di seluruh kelas. Jika terdapat dua kelompok yang memilih tema yang sama, kelompok yang mendaftarkannya terlebih dahulu akan mendapatkan tema tersebut.)*
    b. Untuk tugas pada modul-modul berikutnya, kalian akan bekerja bersama anggota kelompok di rumah untuk mengembangkan project berdasarkan tema tersebut.
    c. Untuk melihat kontribusi setiap anggota kelompok, pastikan kalian bekerja bersama menggunakan lingkungan GitHub agar kami dapat melihat bahwa beban kerja dan tugas telah dibagi secara adil kepada semua anggota (tidak ada satu orang yang mengerjakan semuanya sendirian).
    d. Tugas ini akan berlanjut sepanjang Modul 1–6. Kalian akan terus mengembangkan project yang sama dari modul sebelumnya. Pastikan semua modul sebelumnya dapat berjalan dengan baik karena semuanya juga akan diperiksa pada akhir perkuliahan.

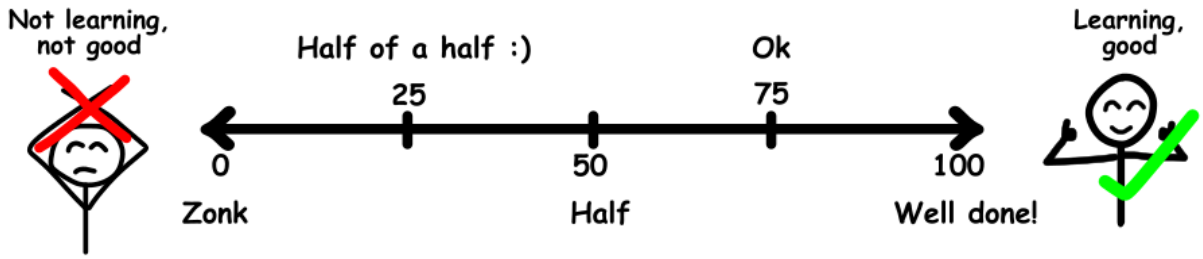
2. **Buat aplikasi kalian bersama:**
    a. Tugas kalian adalah menerapkan semua yang telah dipelajari dalam modul ini ke dalam satu aplikasi yang dapat berjalan **(perhatikan kriteria penilaian dengan saksama).**
    b. Pastikan aplikasi kalian mengikuti tema yang telah dipilih.
    c. Kalian bebas menerapkan materi yang telah dipelajari di bagian mana pun dalam kode. Pastikan penerapannya masuk akal dan dilakukan dengan benar.
    d. Kalian tidak perlu melakukan *build* aplikasi. Namun, jika ingin melakukannya, tentu saja diperbolehkan. Kalian juga dapat menggunakan Expo Go dengan menjalankan: npx expo start --go.
    e. Saya menyediakan contoh aplikasi dasar berikut: Tautan (akan di up setelah masuk pekan materi modul 3).

3. **Di pekan demo, kalian perlu:**
    a. Menunjukkan hasil pekerjaan kalian kepada asisten praktikum **(secara individu).**
    b. Menjawab pertanyaan yang diberikan oleh asisten praktikum **(secara individu).**
    c. Melakukan modifikasi kode secara langsung sesuai permintaan asisten praktikum **(secara individu).**
    d. Praktikan tidak diperkenankan membuka modul atau menggunakan fitur AI pada code editor (inline maupun chatbot agent).

2. KRITERIA PENILAIAN

Assessment Criteria	Assessment Weight
Codelab 1	**40%**
Tugas Pekan Demo	**60%**
Mengambil data dari External API dan memasukkannya ke dalam Object menggunakan Interface.	10%
Menggunakan Custom & Pre-built Async Function.	10%
Menggunakan fungsi .then(), .resolve(), dan .reject().	10%
Menjawab Pertanyaan.	15%
Menyelesaikan Modifikasi Kode.	15%
TOTAL	**100%**

3. SKALA PENILAIAN



KESIMPULAN AKHIR MODUL

Pada *codelab* di modul ini, kamu hanya perlu menyalin dan menempelkan kode yang sudah saya siapkan, *benar?* Kamu tidak perlu mengetahui proses berpikir di baliknya—alur logika, pemecahan masalah, penyusunan kode, dan sebagainya.

Namun, pada modul berikutnya **(terutama Modul 5)**, saya akan mengajarkan cara melakukan semuanya sendiri. Mulai dari menemukan ide, membuat daftar tugas tentang apa yang harus dikerjakan terlebih dahulu, melakukan *debugging* saat terdapat bug, membaca dokumentasi, dan sebagainya. Dengan begitu, kamu akan memiliki keterampilan yang dibutuhkan untuk menjadi developer profesional.

Memang, kita dapat menggunakan **AI** untuk membuat program sederhana. Namun, ketika sebuah project mulai berkembang dan menjadi lebih kompleks, project tersebut membutuhkan tim yang bertugas melakukan *maintenance*, dokumentasi struktur project yang baik, serta *workflow* yang efisien untuk mendukung pengembangan lebih lanjut.

Jika kamu hanya memiliki pola pikir ***"kalau kodenya sudah berjalan, jangan disentuh,"*** kamu tidak akan mampu bertahan di lingkungan tersebut. Kamu perlu benar-benar memahami apa yang sedang kamu kerjakan.

Jadi, tidak masalah menggunakan **AI**, tetapi setidaknya kamu harus memahami **proses berpikir di balik program tersebut sejak awal**. Oke? Kita akan membahas juga bagaimana cara menerapkan konsep vibe coding di modul mendatang!